Kubernetes: первое приложение

Занятие 05 · День 3, утро

От пустого namespace до масштабируемого сервиса с обновлениями без простоя

Что сделаем сегодня

1. Разложим приложение на объекты Kubernetes
2. Напишем манифесты: Namespace, ConfigMap, Secret, Deployment, Service
3. Откроем доступ снаружи через **MetalLB**
4. Масштабируем, обновим версию, **откатим**
5. Научимся **отлаживать**: Pending, CrashLoopBackOff, ImagePullBackOff
6. Включим **автомасштабирование** (HPA) и **постоянное хранилище**

Демо-приложение: **podinfo** — маленький веб-сервис, показывающий версию, имя пода и настройки.

Из чего состоит приложение

Требование	Объект Kubernetes
Изолированное место	`Namespace demo`
Настройки (цвет, сообщение)	`ConfigMap`
Пароль / токен	`Secret`
N копий, обновление без простоя	`Deployment` + пробы + ресурсы
Адрес внутри кластера	`Service` ClusterIP
Адрес снаружи	`Service` LoadBalancer (MetalLB)
Масштаб по нагрузке	`HorizontalPodAutoscaler`
Данные, переживающие под	`PersistentVolumeClaim`

ConfigMap и Secret

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: podinfo-config
  namespace: demo
data:
  PODINFO_UI_MESSAGE: "Привет от DevOps-курса!"
  PODINFO_UI_COLOR: "#326CE5"
---
apiVersion: v1
kind: Secret
metadata:
  name: podinfo-secret
  namespace: demo
type: Opaque
stringData:                    # stringData — открытым текстом, API сам закодирует в base64
  API_TOKEN: "demo-token-123"
```

Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: podinfo
  namespace: demo
  labels: { app.kubernetes.io/name: podinfo }
spec:
  replicas: 2
  selector:
    matchLabels: { app.kubernetes.io/name: podinfo }
  template:
    metadata:
      labels: { app.kubernetes.io/name: podinfo }
    spec:
      containers:
        - name: podinfo
          image: ghcr.io/stefanprodan/podinfo:6.7.0
          ports: [{ name: http, containerPort: 9898 }]
          envFrom:
            - configMapRef: { name: podinfo-config }
            - secretRef:    { name: podinfo-secret }
          resources:
            requests: { cpu: 50m, memory: 32Mi }
            limits:   { memory: 128Mi }
          readinessProbe: { httpGet: { path: /readyz, port: http }, periodSeconds: 5 }
          livenessProbe:  { httpGet: { path: /healthz, port: http }, initialDelaySeconds: 5 }
```

Service: внутри и снаружи

```yaml
apiVersion: v1
kind: Service
metadata:
  name: podinfo
  namespace: demo
spec:
  type: LoadBalancer           # MetalLB выдаст IP из 192.168.56.200–220
  selector: { app.kubernetes.io/name: podinfo }
  ports:
    - name: http
      port: 80
      targetPort: http         # можно ссылаться на имя порта контейнера
```

```
kubectl apply -f k8s/
kubectl -n demo get svc podinfo     # EXTERNAL-IP
curl http://192.168.56.201/
```

Откройте адрес в браузере на своём компьютере — сеть стенда доступна с хоста.

ДЕМО Демо: деплой и проверка

```
kubectl apply -f k8s/
kubectl -n demo rollout status deploy/podinfo
kubectl -n demo get pods -o wide             # на каких узлах?
kubectl -n demo get endpointslices           # IP подов за Service

# изнутри кластера — по DNS-имени
kubectl -n demo run curl --rm -it --image=curlimages/curl:8.11.1 -- \
  curl -s http://podinfo.demo.svc.cluster.local/

# балансировка: имя пода меняется
for i in $(seq 5); do curl -s http://<EXTERNAL-IP>/ | grep hostname; done
```

Масштабирование

```
kubectl -n demo scale deploy/podinfo --replicas=4      # быстро, но мимо Git
```

Правильно — изменить `replicas` в файле и `kubectl apply`.

- Новые поды попадают в Service **только после readinessProbe**
- Scheduler разносит реплики по узлам
- Не хватает ресурсов → часть подов в `Pending` (смотрим `describe` → Events)

```
kubectl -n demo get pods -o wide -w
kubectl describe node worker-1 | grep -A8 "Allocated resources"
```

ДЕМО Обновление версии без простоя

```
# в одном терминале — непрерывные запросы
while true; do curl -s http://<EXTERNAL-IP>/version; echo; sleep 0.3; done

# в другом — новая версия
kubectl -n demo set image deploy/podinfo podinfo=ghcr.io/stefanprodan/podinfo:6.7.1
kubectl -n demo rollout status deploy/podinfo
kubectl -n demo get rs                       # старый ReplicaSet с 0 подов остался
kubectl -n demo rollout history deploy/podinfo
```

Ответы плавно переключаются с `6.7.0` на `6.7.1` — **ни одной ошибки**: RollingUpdate + readinessProbe.

Откат

```
kubectl -n demo rollout undo deploy/podinfo                 # на предыдущую ревизию
kubectl -n demo rollout undo deploy/podinfo --to-revision=1 # на конкретную
kubectl -n demo rollout pause deploy/podinfo                # пауза — для серии правок
kubectl -n demo rollout resume deploy/podinfo
```

- Откат = переключение на старый **ReplicaSet** (`revisionHistoryLimit`, по умолчанию 10)
- Пишите причину изменения:
  `kubectl annotate deploy/podinfo kubernetes.io/change-cause="6.7.1: fix"`
- В реальной жизни откат делают через **Git / CI / Helm** — чтобы файлы и кластер не расходились

Отладка: алгоритм

1. `kubectl get pods` — какой **статус**?
2. `kubectl describe pod <под>` — **Events** внизу: что пошло не так
3. `kubectl logs <под>` (`--previous` — лог упавшего контейнера)
4. `kubectl get events -n demo --sort-by=.lastTimestamp`
5. Внутрь: `kubectl exec -it <под> -- sh` или `kubectl debug`
6. Сеть: `kubectl get endpointslices` — есть ли поды за Service?

Типичные поломки

Статус / симптом	Частая причина	Где смотреть
`Pending`	не хватает CPU/памяти, нет PV, taint	`describe pod` → Events
`ImagePullBackOff`	опечатка в образе/теге, приватный registry без `imagePullSecrets`	`describe pod`
`CrashLoopBackOff`	приложение падает: нет переменной, неверная команда	`logs --previous`
`OOMKilled`	превышен `limits.memory`	`describe pod` → Last State
Running, но 0/1 Ready	readinessProbe не проходит: путь/порт	`describe pod`
Service не отвечает	selector не совпадает с метками → пустые endpoints	`get endpointslices`

ДЕМО Демо: ломаем и чиним

```
# 1. несуществующий тег
kubectl -n demo set image deploy/podinfo podinfo=ghcr.io/stefanprodan/podinfo:9.9.9
kubectl -n demo get pods ; kubectl -n demo describe pod <новый-под> | tail -8
kubectl -n demo rollout undo deploy/podinfo

# 2. неверный порт readiness
kubectl -n demo patch deploy podinfo --type=json -p \
 '[{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/httpGet/port","value":9999}]'
kubectl -n demo get pods          # 0/1 Running — старые поды продолжают работать!
kubectl -n demo rollout undo deploy/podinfo

# 3. слишком большой запрос памяти
kubectl -n demo set resources deploy/podinfo --requests=memory=20Gi
kubectl -n demo describe pod <Pending-под> | grep -A3 Events
```

Автомасштабирование: HPA

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: podinfo, namespace: demo }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: podinfo }
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 60 }   # % от requests.cpu
```

- Нужен **metrics-server** (включили в Kubespray) и **requests.cpu** у контейнера
- `kubectl -n demo get hpa -w` + нагрузка — смотрим, как растёт число реплик
- С HPA поле `replicas` в Deployment лучше **не задавать** — иначе apply будет «сбрасывать» масштаб

Данные: PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: podinfo-data, namespace: demo }
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 1Gi } }
---
# в Deployment
      volumes:
        - name: data
          persistentVolumeClaim: { claimName: podinfo-data }
      containers:
        - name: podinfo
          volumeMounts: [{ name: data, mountPath: /data }]
```

- **local-path**: том — каталог `/opt/local-path-provisioner/…` на узле; под «привязывается» к узлу
- RWO-том + несколько реплик на разных узлах = проблема → для БД — **StatefulSet**

Чек-лист «production-ready» приложения

- [x] Манифесты в Git, применяются через `apply` / CI
- [x] Фиксированный тег образа
- [x] `requests` / `limits`, readiness/liveness
- [x] ≥ 2 реплики, RollingUpdate с `maxUnavailable: 0`
- [x] Конфигурация в ConfigMap, секреты в Secret (и не в Git)
- [x] Метки `app.kubernetes.io/*`
- [ ] Свой образ в своём registry — **день 4–5**
- [ ] Автоматический деплой из GitLab CI — **день 5**

Итоги

- Приложение = набор связанных объектов, связанных **метками**
- **RollingUpdate + readinessProbe** = обновление без простоя
- `rollout undo` — быстрый откат, но источник правды — **Git**
- Отладка: `get` → `describe` (Events) → `logs` → `exec`
- HPA требует metrics-server и requests; PVC — StorageClass

Практика 05

Своё приложение в кластере

Задание — на платформе testlms.kz